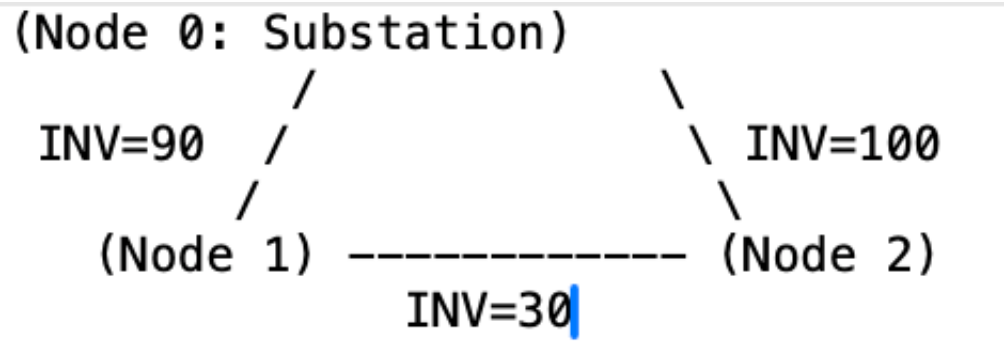
Double-click (or enter) to edit

Warm Up: n=2, T=2, perfect anti-correlation



- Time Periods (T): 2
- Generators (N): 2
- Random Seeds: None used (deterministic LP).
- Generation Matrix: g 1 =[0,1], g 2 =[1,0]
- Investment Costs (INV l): INV (1,0) =90, INV (2,0) =100, INV (1,2) =30 ($/MW.year)

```
import Pkg;
Pkg.add("JuMP")
Pkg.add("HiGHS")
Pkg.add("StatsPlots")
using JuMP
using HiGHS
using Plots
using Printf
using StatsPlots

# --- 1. COALITION-COST LP ---
function compute_coalition_cost(S, g, INV)
    model = Model(HiGHS.Optimizer)
    set_silent(model)
```

```
    nodes = [1, 2]
    edges = [(1,0), (2,0), (1,2)]
    T = 1:2

    @variable(model, cap[edges] >= 0)
    @variable(model, f_dir[edges, T] >= 0)
    @variable(model, f_rev[edges, T] >= 0)

    @objective(model, Min, sum(INV[e] * cap[e] for e in edges))

    for t in T
        for i in nodes
            gen_val = (i in S) ? g[i][t] : 0.0
            inflow  = sum(f_dir[e,t] for e in edges if e[2]==i; init=0.
                      sum(f_rev[e,t] for e in edges if e[1]==i; init=0.
            outflow = sum(f_dir[e,t] for e in edges if e[1]==i; init=0.
                      sum(f_rev[e,t] for e in edges if e[2]==i; init=0.
            @constraint(model, gen_val + inflow == outflow)
        end
        for e in edges
            @constraint(model, f_dir[e,t] <= cap[e])
            @constraint(model, f_rev[e,t] <= cap[e])
        end
    end

    optimize!(model)
    return objective_value(model)
end

# --- 2. NUCLEOLUS (Sequential LP) ---
function compute_nucleolus(C_dict, N_players)
    all_coalitions = [S for S in keys(C_dict) if 0 < length(S) < length
    active_coalitions = copy(all_coalitions)
    fixed_excesses = Dict()
    x_alloc = Dict(i => 0.0 for i in N_players)

    while !isempty(active_coalitions)
        model = Model(HiGHS.Optimizer)
        set_silent(model)
        @variable(model, x[N_players])
        @variable(model, e)
        @constraint(model, sum(x[i] for i in N_players) == C_dict[N_pla
        for S in all_coalitions
            if haskey(fixed_excesses, S)
```

```
                    @constraint(model, C_dict[S] - sum(x[i] for i in S) ==
                else
                    @constraint(model, C_dict[S] - sum(x[i] for i in S) >=
                end
            end
            @objective(model, Max, e)
            optimize!(model)
            e_val   = value(e)
            x_val   = value.(x)

            newly_fixed = []
            for S in active_coalitions
                test = Model(HiGHS.Optimizer)
                set_silent(test)
                @variable(test, tx[N_players])
                @constraint(test, sum(tx[i] for i in N_players) == C_dict[N
                for S2 in all_coalitions
                    if haskey(fixed_excesses, S2)
                        @constraint(test, C_dict[S2] - sum(tx[i] for i in S
                    else
                        @constraint(test, C_dict[S2] - sum(tx[i] for i in S
                    end
                end
                @objective(test, Max, C_dict[S] - sum(tx[i] for i in S))
                optimize!(test)
                objective_value(test) <= e_val + 1e-5 && push!(newly_fixed,
            end

            for S in newly_fixed
                fixed_excesses[S] = e_val
                filter!(s -> s != S, active_coalitions)
            end
            for i in N_players
                x_alloc[i] = x_val[i]
            end
        end
        return x_alloc
    end

    # --- DATA ---
    g   = Dict(1 => [0.0, 1.0], 2 => [1.0, 0.0])
    INV = Dict((1,0) => 90.0, (2,0) => 100.0, (1,2) => 30.0)

    C_dict = Dict(
        [1]    => compute_coalition_cost([1],    g, INV),
```

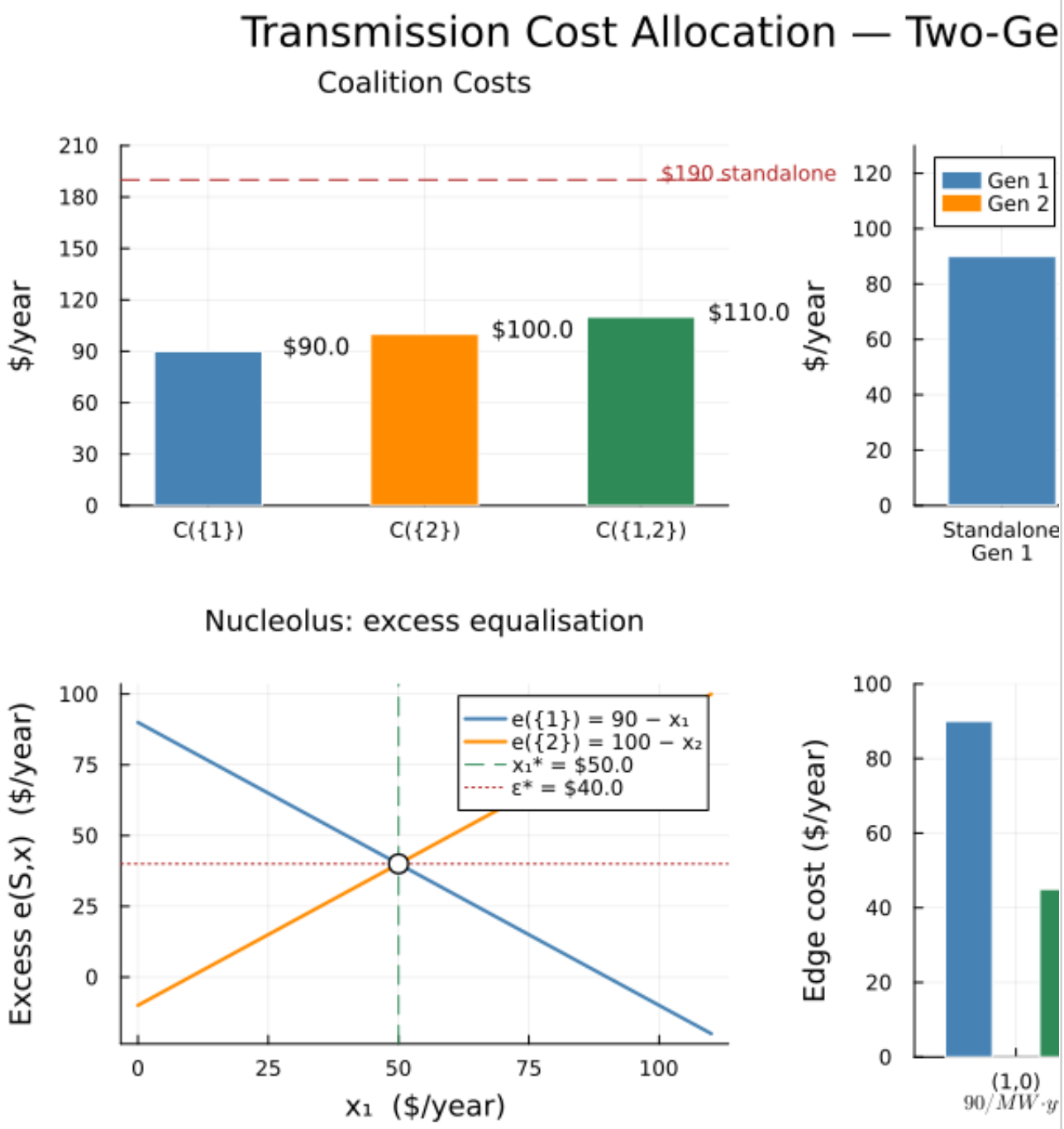
```
    [2]    => compute_coalition_cost([2],    g, INV),
    [1,2]  => compute_coalition_cost([1, 2], g, INV)
)

x_star = compute_nucleolus(C_dict, [1, 2])
CN     = C_dict[[1,2]]
x1, x2 = x_star[1], x_star[2]

# --- PRINT TABLE ---
println("="^50)
println("  Coalition Costs")
println("="^50)
println("  C({1})   = \$($(C_dict[[1]]))/yr")
println("  C({2})   = \$($(C_dict[[2]]))/yr")
println("  C({1,2}) = \$($(CN))/yr  [grand coalition]")
println()
println("  Standalone sum = \$($(C_dict[[1]] + C_dict[[2]]))/yr")
println("  Total savings  = \$($(C_dict[[1]] + C_dict[[2]] - CN))/yr")
println()
println("="^50)
println("  Nucleolus Allocation x*")
println("="^50)
@printf("  x*(Gen 1) = \$%6.2f/yr  (%5.1f%%)\n", x1, 100*x1/CN)
@printf("  x*(Gen 2) = \$%6.2f/yr  (%5.1f%%)\n", x2, 100*x2/CN)
@printf("  Sum       = \$%6.2f/yr  (= C(N) ✓)\n", x1+x2)
println()
@printf("  Gen 1 saves: \$%.2f/yr  (%.1f%% reduction)\n", C_dict[[1]]-x
@printf("  Gen 2 saves: \$%.2f/yr  (%.1f%% reduction)\n", C_dict[[2]]-x
eps_star = C_dict[[1]] - x1
@printf("  Min excess ε* = \$%.2f/yr (equal for both coalitions)\n", ep
println("="^50)

# --- PLOT 1: Coalition Costs ---
p1 = bar(
    ["C({1})", "C({2})", "C({1,2})"],
    [C_dict[[1]], C_dict[[2]], CN],
    color       = [:steelblue, :darkorange, :seagreen],
    legend      = false,
    ylabel      = "\$/year",
    title       = "Coalition Costs",
    ylims       = (0, 210),
    bar_width   = 0.5,
    linecolor   = :white
)
hline!(p1, [C_dict[[1]] + C_dict[[2]]],
```

```
       linestyle=:dash, color=:firebrick, label="Standalone sum")
annotate!(p1, 3.0, C_dict[[1]]+C_dict[[2]]+4,
          text("\$$(Int(C_dict[[1]]+C_dict[[2]])) standalone", 8, :fire
for (xi, v) in enumerate([C_dict[[1]], C_dict[[2]], CN])
    annotate!(p1, xi, v+3, text("\$$v", 9, :black, :center))
end

# --- PLOT 2: Nucleolus Allocation (stacked) ---
p2 = groupedbar(
    ["Standalone\nGen 1", "Standalone\nGen 2", "Coalition\nAllocation"]
    [C_dict[[1]] 0; 0 C_dict[[2]]; x1 x2],
    bar_position = :stack,
    color        = [:steelblue :darkorange],
    label        = ["Gen 1" "Gen 2"],
    ylabel       = "\$/year",
    title        = "Nucleolus x*",
    ylims        = (0, 130),
    bar_width    = 0.5,
    linecolor    = :white
)
annotate!(p2, 3, x1/2,    text("\$$x1", 9, :white, :center))
annotate!(p2, 3, x1+x2/2, text("\$$x2", 9, :white, :center))

# --- PLOT 3: Excess curves ---
x_range = 0:0.5:CN
exc1 = C_dict[[1]] .- x_range
exc2 = C_dict[[2]] .- (CN .- x_range)
p3 = plot(x_range, exc1,
    label     = "e({1}) = $(Int(C_dict[[1]])) - x₁",
    color     = :steelblue, lw=2,
    xlabel    = "x₁  (\$/year)",
    ylabel    = "Excess e(S,x)  (\$/year)",
    title     = "Nucleolus: excess equalisation",
    legend    = :topright
)
plot!(p3, x_range, exc2,
    label = "e({2}) = $(Int(C_dict[[2]])) - x₂",
    color = :darkorange, lw=2)
vline!(p3, [x1],    linestyle=:dash,   color=:seagreen, label="x₁* = \$
hline!(p3, [eps_star], linestyle=:dot, color=:firebrick, label="ε* = \$
scatter!(p3, [x1], [eps_star], color=:white, markerstrokecolor=:black,

# --- PLOT 4: Edge capacity breakdown ---
edge_names   = ["(1,0)\n\$90/MW·yr", "(2,0)\n\$100/MW·yr", "(1,2)\n\$30
solo1_costs  = [90.0, 0.0, 0.0]
```

```
solo2_costs  = [0.0, 100.0, 0.0]
gc_costs     = [90*0.5, 100*0.5, 30*0.5]   # cap=0.5 MW each at optimum
p4 = groupedbar(
    edge_names,
    hcat(solo1_costs, solo2_costs, gc_costs),
    label      = ["Standalone Gen 1" "Standalone Gen 2" "Grand Coalitio
    color      = [:steelblue :darkorange :seagreen],
    ylabel     = "Edge cost (\$/year)",
    title      = "Capacity cost by edge",
    bar_width  = 0.7,
    linecolor  = :white,
    legend     = :topright
)

# --- COMBINE & DISPLAY ---
plot(p1, p2, p3, p4,
    layout      = (2, 2),
    size        = (900, 650),
    plot_title  = "Transmission Cost Allocation — Two-Generator Network
    margin      = 5Plots.mm,
    titlefont   = font(11)
)
```

```
 Resolving package versions...
   Project No packages added to or removed from `~/.julia/environments/
  Manifest No packages added to or removed from `~/.julia/environments/
 Resolving package versions...
   Project No packages added to or removed from `~/.julia/environments/
  Manifest No packages added to or removed from `~/.julia/environments/
 Resolving package versions...
   Project No packages added to or removed from `~/.julia/environments/
  Manifest No packages added to or removed from `~/.julia/environments/
================================================
Coalition Costs
================================================
C({1})   = $(90.0)/yr
C({2})   = $(100.0)/yr
C({1,2}) = $(110.0)/yr  [grand coalition]

Standalone sum = $(190.0)/yr
Total savings  = $(80.0)/yr

================================================
Nucleolus Allocation x*
================================================
x*(Gen 1) = $ 50.00/yr  ( 45.5%)
x*(Gen 2) = $ 60.00/yr  ( 54.5%)
Sum       = $110.00/yr  (= C(N) ✓)
```

```
Gen 1 saves: $40.00/yr  (44.4% reduction)
Gen 2 saves: $40.00/yr  (40.0% reduction)
Min excess ε* = $40.00/yr (equal for both coalitions)
================================================
```



Double-click (or enter) to edit